Algebraic Effect

Behind React: leverage **side effect**

6/10/2023 a@yuzhes.com 代数效应

Accessibility

- Slides & Code are available at yuzhes.com
-

Table of Contents

- Introduce Algebraic Effect
- Why we need Algebraic Effect
- <Suspense /> in React
- Implement our own Algebraic Effect function

Sync Get Name

```
1  const getUserName = (userId) => doSthToGetUserName(userId);
2
3  const main = () => {
4    const userName = getUserName(123);
5    console.log(userName);
6  };
```

Async Get Name

```
1  const getUserName = async (userId) => (await axios.get(`${userId}`)).data;
2
3  const main = async () => {
4    const userName = await getUserName(123);
5    console.log(userName);
6  };
```

Stop ... and resume

- perform a effect, let previous function stop, and resume when effect done.

```
const getUserName = (id) => {
    try {
        const uname = perform 'user_name';
    } handle (effect) {
        if (effect === 'user_name') {
            const resp = await fetch(`uname/${id}`);
            resume resp;
        }
    }
}
```

```
const main = () => {
  const userName = getUserName(123);
  console.log(userName);
};
```

Behind this

throw V.S. resume

- When throw:
  - function call stack & local vars are GC-ed.

- When resume(call a effect):
  - construct a callback, and let `resume` to call it.

Compare

```
1  const getUserName = (userId) => doSthToGetUserName(userId);
2
3  const main = () => {
4    const userName = getUserName(123);
5    console.log(userName);
6  };
```

- interrupt -> getUserName -> resume

Compare

```
1  const getUserName = async (userId) => (await axios.get(`${userId}`)).data;
2
3  const main = async () => {
4    const userName = await getUserName(123);
5    console.log(userName);
6  };
```

- Async function will not block, **it self won't do interrupt**
- So have to use `await` to explicitly require an await interrupt.
- So the higher level function `main` has do async interrupt.

Compare

```
const getUserName = (id) => {
    try {
        const uname = perform 'user_name';
    } handle (effect) {
        if (effect === 'user_name') {
            const resp = await fetch(`uname/${id}`);
            resume resp;
        }
    }
}
```

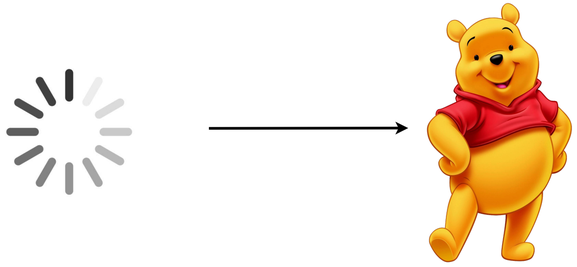
- The logic of `try ... resume ...` is to "interrupt and resume"
- we don't need other ways to interrupt the procedure.

Summary

- We used "try ... resume ..." to avoid pollution of async&await.
- We have async code to be written in a sync schema.

<Suspense />

- When you are doing something async in Suspense, it fallback.



- <Suspense fallback={加载中}>
  - {fetch 小熊维尼（200MB）}
- <Suspense>

What happened?

- your async code is warped by `react-cache`
- react cache instantly **throw** your promise, React will catch this "error". So Suspense knows that it's loading.
- this promise is caught, when resolve, Suspense knows it's done.

```
function LazyComponent() {
    const data = fetchDataFromAPI(); // synchronous call
    return <div>{data}</div>;
}

function App() {
    return (
        <div>
            <Suspense fallback={<div>Loading...</div>}>
                <LazyComponent />
            </Suspense>
        </div>
    );
}
```

why cache in its name?

- when promise is resolved, we cache the result.
- we call the previous function again, and it hit cache, and immediately re-render the proper image.

Do it yourself

- By retrying.
- main is called twice.
- The first time: main() is called, cache miss, call fetch() (but not waiting it), and instantly throw this promise.
- run() catch this promise, and add a callback to it: <u>When fetch() finished, cache the result, call main().</u>
- after fetch() done, main() is called again, cache hit.

```
function run(func) {
  try {
    func()
  } catch (e) {
    if (e instanceof Promise) {
      e.then((ret: any) => {
        func()
      })
    }
  }
}
function main() {
  if (cache) {
    return cache
  } else {
    throw fetch('something')
  }
}
```

```
async function fetch(s) {
  return new Promise(resolve => {
    setTimeout(() => {
      cache = 'something'
      resolve()
    }, 2000);
  });
}

run(main)
```

```
import React from 'react';

function ThankYou() {
  return (
    <div>
   <Suspense fallback={<div>Loading...</div>}>
          <p>Thank you for listening!</p>
      </Suspense>
    </div>
  );
}

export default ThankYou;
```

Reference

- https://overreacted.io/algebraic-effects-for-the-rest-of-us/
- https://mongkii.com/blog/2021-05-08-talk-about-algebraic-effects

Discarded Pages

Pure Function

y=2x

- if the return value is determined by its input values only and the return value is always the same for the same input values.

- Keys:
  - No access to outer functions / variables (context independent)
  - Same input -> same output
  - No async (no await)

Why pure function matters?

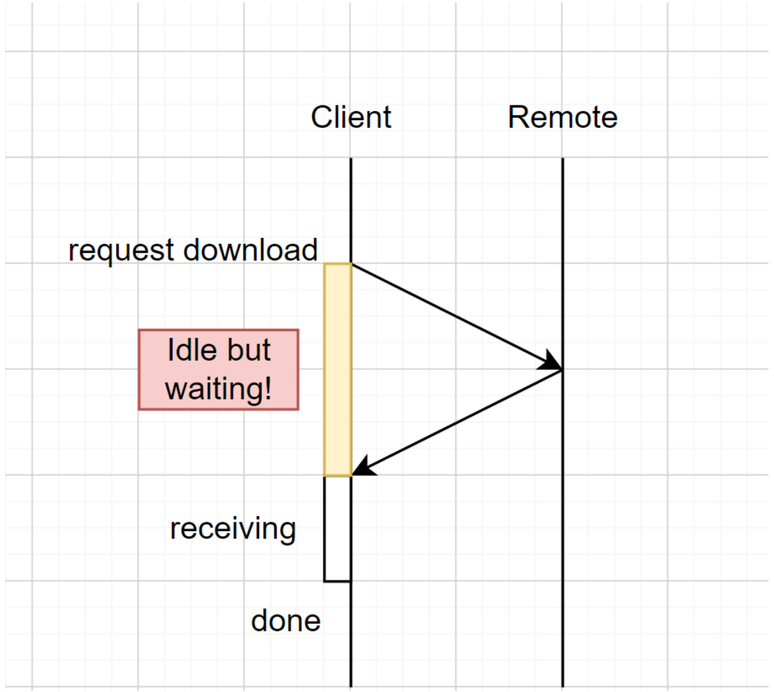
JavaScript is single-threaded model

- We are not talking about `worker` (multi-threading API)
- Parsing HTML, executing JS ... All in one thread.
- Event loop is introduced to tackle the issue that there is no `multi-threading` in JavaScript.

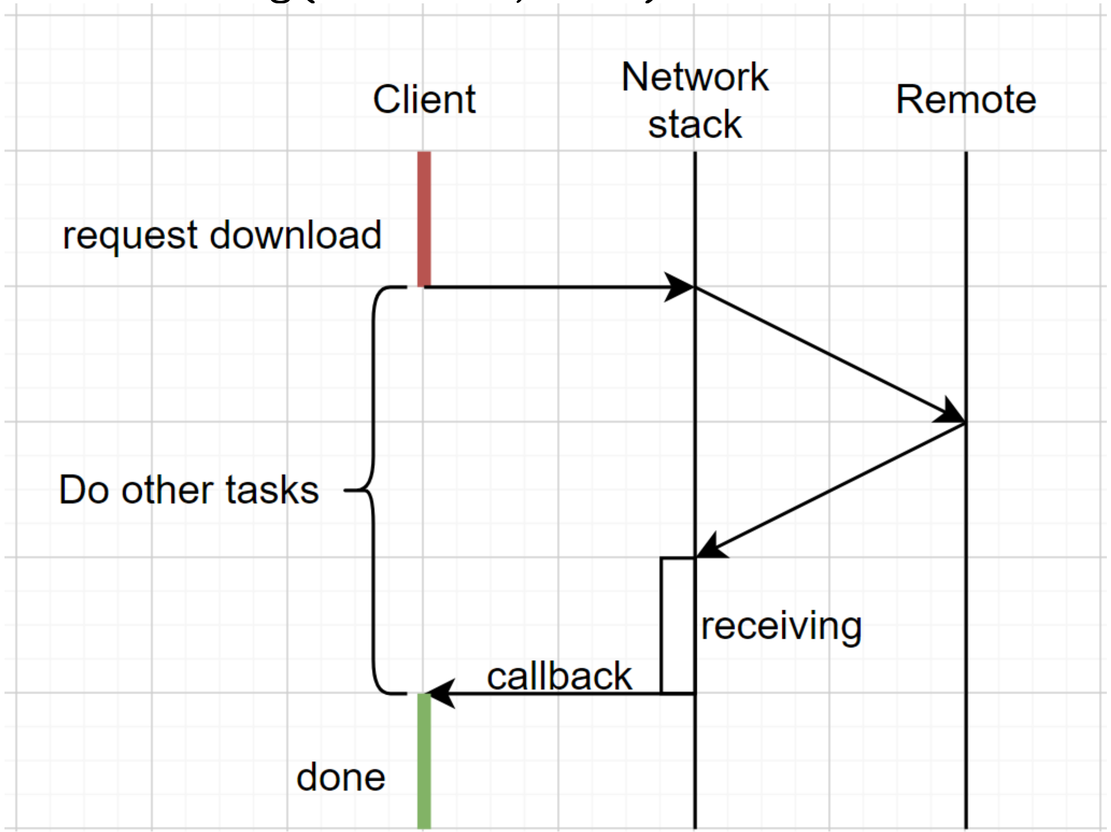
Event loop

imaging a file downloading scenario

```
def downloadSync(url):
    establish connection to this url
    download(url)
    say `done!`
```




```
def downloadAsync(url):
    requestDownload(url, whenDone: say `done!`)
    #more JS style:
    requestDownload(url, (res)=>{
        console.log(`done!`, res)
    })
```



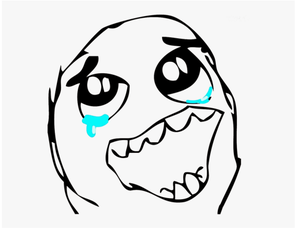
Keep UI free of side effects

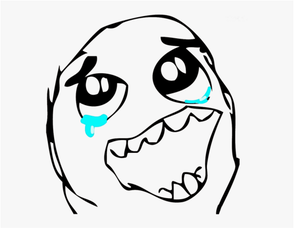
- Do not let UI wait for anything!
- Where there is a waiting, there is an async.

Not always the case

what if fetch something to display?

```
main() {
     download(data)
     display(data)
}
```





```
main() {
     displayLoading()
     requestDownload(data).then(
          hideLoading()
          display(data)
     )
} // won't stuck
```

images from pngitems & fandom

Suspend

- enables components to suspend rendering to wait for a particular operation to complete.
- React Suspense works by throwing a Promise returned by a component. React will take over the rendering of the component and display a fallback UI until the data is resolved.

```
function LazyComponent() {
    const data = fetchDataFromAPI(); // synchronous call
    return <div>{data}</div>;
}

function App() {
    return (
        <div>
            <Suspense fallback={<div>Loading...</div>}>
                <LazyComponent />
            </Suspense>
        </div>
    );
}
```

Our try ... resume is not here yet
(may be in ES 2025)

- But React do make async sync. Let's check it out.
- We implemented the same trick as React source code do

Code Availability

- Code is available at
- https://pastebin.com/1bTBVy0M

really "Transformed" async into sync?

- Not really.
- We separate side effect, and realize the function of a async call by calling a sync function.

Good?

Yes and no. Still needs discussing

- goto -> if for while
- callback / promise -> async await
- (possibly) try ... catch ... -> try ... resume ...

Ending

- How we design a programing language, a framework, etc. is a endless, dynamic journey of discovery and testing.
- For everyone of us:
  - Learning
  - Thinking
  - Doing